



S5/b Consensus Testing Summary

Verification Evidence for the S5/b Testing-Mode Implementation

Rincoin Community Forge

2026-08-25

Table of contents

1 Purpose of This Document	2
2 What Changes at H1, Briefly	2
3 Summary	3
4 Methods: What Was Tested, and How	4
4.1 Unit-level	4
4.2 Single-node functional	5
4.3 Multi-node functional	5
4.4 Cross-implementation	5
5 Results: Did It Pass?	5
5.1 Unit-level — pass	5
5.2 Single-node functional — pass	6
5.3 Multi-node functional — pass	6
5.4 Cross-implementation — partially done, passing so far	7
6 Scope: What This Testing Does Not Cover	7
7 Provenance	7

 Discussion Draft

Document version 0.1 · 2026-08-25

No scenario has been selected, preferred, or adopted. This document reports testing evidence for a **testing-mode-only** implementation of the S5/b candidate; it does not argue for S5/b over S1 or S6/b, and it is not a readiness or deployment claim.

 Editorial note — for maintainers, not part of the reported evidence

This branch is pushed to `rincoin-community/rincoin-core`; CI was still in progress at time of writing (see Provenance below for the run). All results below are from local verification, not from that CI run concluding. The cross-implementation producer $\times$ validator matrix and two narrow commitment test-vector cases (inherited from S1, not new here) are also outstanding. Every one of these is called out explicitly, in its own place, below. The full, current list is [S5B/open-items.md](#).

1 Purpose of This Document

This is a management summary, not a specification. It reports whether the S5/b testing-mode implementation actually behaves as `technology/consensus-transition.md` and `analysis/Rincoin_840k_S5B_Consensus_Change_Specification.qmd` say it should — checked by running it, not by re-reading the design. The full evidence — every test case, raw logs, exact commits — is in [S5B/](#), not repeated here.

This scenario's testing-mode branch was built by branching directly from the already-tested `consensus/s1-testing` branch and reusing its shared infrastructure (coinbase commitment, `sig_fork_id`, mainnet blocker, wallet/ PSBT signing) unmodified — confirmed scenario-agnostic by direct code review before branching, not assumed. Only the subsidy formula and a handful of identity constants changed. This document's structure mirrors [S1's own summary](#) directly, not as a separate design exercise.

2 What Changes at H1, Briefly

At block height 840,000 (mainnet), three things change together: the block subsidy schedule, a scheduled coinbase commitment identifying the chosen schedule, and a sighash-level replay identifier (`sig_fork_id`) mixed into new transaction signatures. The commitment and `sig_fork_id` mechanisms are identical across every candidate scenario; only the subsidy schedule differs. S5/b's schedule: an extended, 10x-longer halving epoch, phase-anchored to height 840,000 — `nSubsidyHalvingInterval` (630,000 on mainnet — the pre-activation epoch boundary, *not* the activation height itself), then plain binary halving per extended epoch, no floor, no tail. Full rationale and byte-level specification: `analysis/Rincoin_840k_S5B_Consensus_Change_Specification.qmd`. This document assumes that design and tests it; it does not re-argue it.

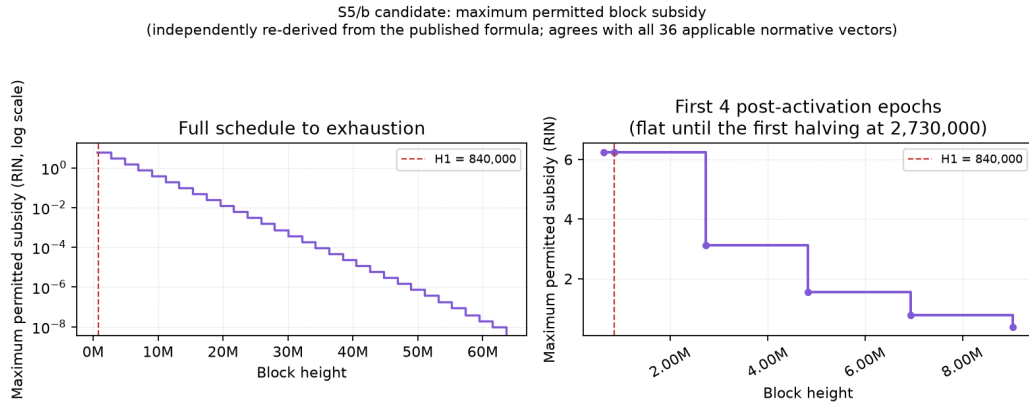


Figure 1: Maximum permitted block subsidy under the S5/b candidate, independently re-derived from the published formula (not imported from the implementation under test) and confirmed to agree with all 36 applicable rows of `analysis/data/S5B_normative_test_vectors.csv`, the specification’s own conformance vector file.

3 Summary

Every test that has been run passes. 532 of 532 C++ unit tests, 159 of 159 functional tests (the fork-specific suite plus the full pre-existing CI allowlist, unmodified), and every cross-implementation comparison run so far. One real defect in the *test suite itself* (not the consensus implementation) was found and fixed while adapting it for this scenario’s structurally different epoch length — reported in Results as part of the evidence, not as a caveat against it.

Evidence layer	Result	Status
C++ unit tests	532 / 532	Done
Fork scenario functional tests (single- and two-node)	6 / 6	Done
Full functional CI allowlist (fork + pre-existing, unmodified)	159 / 159	Done
Independent (non-C++) subsidy re-derivation	agrees with all 36 applicable vectors	Done
Cross-implementation producer x validator matrix	not yet run	Outstanding
Fuzz targets (commitment parsing, sig_fork_id sighash mixing)	not yet written	Outstanding
Pool / seeder integration rehearsal	not yet run	Outstanding

Evidence layer	Result	Status
Push to rincoin-community/rincoin-core + CI run	not yet done	Outstanding

Two evidence layers are not yet complete, both inherited from S1 rather than newly discovered here: the full cross-implementation producer $\times$ validator matrix, and two narrow commitment test-vector cases. Neither is scenario-specific — closing them once closes them for S1, S5/b, and S6/b together. See Results §5.4 and [S5B/open-items.md](#).

Read **Methods** below for what “tested” means in each row above and why it was tested that way; read **Results** for the detail behind each pass.

4 Methods: What Was Tested, and How

Four groups, used consistently through both this section and Results — the same four used for S1, since the mechanism being tested is identical:

Group	What it checks	Why it’s a separate group
Unit-level	Each piece of consensus logic (subsidy formula, commitment parsing, sighash mixing) against frozen, byte-exact vectors	Smallest testable unit; catches an error before it can reach anything that depends on it
Single-node functional	The same rules, enforced end-to-end by a real running node via RPC/P2P	Unit tests exercise logic in-process; this confirms the rules hold when driven the way a real deployment would drive them
Multi-node functional	Whether the network prefers higher work only when that chain is also rule-valid	This is the entire reason the coinbase-commitment mechanism exists — it has to be checked directly, not inferred from the rule existing
Cross-implementation	What happens when this build meets software that disagrees with it	The only layer that reflects the network as it actually is, not just this codebase

4.1 Unit-level

The subsidy formula is checked against frozen vectors from two independent sources agreeing: `analysis/data/S5B_normative_test_vectors.csv` (the file the specification itself names as the conformance target) and a from-scratch Python re-derivation of the published formula (`verification/scripts/independently_derive_s5b_schedule.py`)

that does not import the C++ implementation. The commitment builder/parser and `sig_fork_id` sighash-mixing code are reused unmodified from `consensus/s1-testing` — their own tests already check them against frozen vectors independent of which subsidy scenario is compiled in.

Every pre-existing sighash and script unit test in the codebase is also re-run, unmodified, as a no-regression guard.

4.2 Single-node functional

Driven through a real, running `rincoind` process via its actual RPC and P2P interfaces, not exercised in-process: coinbase-commitment validation (reused unmodified from S1 — the malformation catalog doesn't depend on the subsidy scenario), and subsidy enforcement both at H1 and at the real second phase boundary (H1 + 9 halving-intervals for this scenario, not one — see the Results note on why this needed a real fix, not just a constant change).

4.3 Multi-node functional

Two testing-mode nodes, connected and synced past the activation height, one isolated and used to mine two competing continuations: a longer chain that is also commitment-valid, and a longer chain that is commitment-*invalid*. Both chains are offered back to the network; which one is adopted is the entire test. Reused unmodified from S1 — this mechanism doesn't depend on the subsidy scenario either.

4.4 Cross-implementation

Every accessible node binary — this implementation, older Rincoin releases, and independently developed forks reachable on a shared network — tested as both block *producer* and block *validator* against every other binary.

i Editorial note — for maintainers

Same outstanding item as S1: the general-purpose `test/fork_interop_matrix.py` tool is not yet built. The two narrower, hand-written comparisons (`feature_fork_vs_legacy.py`, `feature_fork_vs_aevust.py`), reused unmodified from S1, stand in for it — their results are reported in Results §5.4.

5 Results: Did It Pass?

5.1 Unit-level — pass

532 of 532 C++ unit tests pass (one unrelated, pre-existing warning, same as S1's own report). The subsidy schedule's two independent sources agree with each other and with the C++ implementation at every height checked:

Height	Vector ID	CSV value (base units)	C++ unit test result	Note
839,999	S5B-001	625,000,000	625,000,000 — match	last pre-activation block
840,000	S5B-002	625,000,000	625,000,000 — match	activation block; phase index 0
2,729,999	S5B-004	625,000,000	625,000,000 — match	last block of long phase 0
2,730,000	S5B-005	312,500,000	312,500,000 — match	first block of long phase 1
6,930,000	S5B-011	78,125,000	78,125,000 — match	crossing below 1 RIN; first block of long phase 3
63,630,000	S5B-036	0	0 — match	first zero-subsidy block

The commitment-parsing vector catalog carries the same two open exceptions S1’s own annex tracks (not new to this scenario): see [S5B/commitment-test-vectors.md](#).

Full detail and reproduction commands: [S5B/unit-test-results.md](#).

5.2 Single-node functional — pass

All commitment-validation and subsidy-enforcement checks pass against a real running node. One genuine test-suite defect was found and fixed here, not in the implementation under test: the inherited “next epoch boundary” check assumed the wrong offset for this scenario’s 10x-longer post-fork epoch, which (once corrected to the real offset) crossed the regtest MWEB activation height and exposed that hand-built test blocks can’t satisfy MWEB’s extension-block requirement. Fixed with a scenario-portable offset constant and a real-miner-based fallback check past MWEB activation — full account in [S5B/unit-test-results.md](#) and [S5B/functional-test-results.md](#).

5.3 Multi-node functional — pass

Both reorg outcomes are correct, reusing S1’s own mechanism test unmodified: the higher-work, commitment-*valid* chain is adopted; the higher-work, commitment-*invalid* chain is not.

5.4 Cross-implementation — partially done, passing so far

The full producer $\times$ validator matrix has not been run. The two narrower comparisons that are done both pass for what they cover, with the same conclusions already established for S1 (this mechanism doesn't depend on the subsidy scenario): S5/b vs. v1.1.0 diverges cleanly at the activation height; S5/b vs. v1.0.1 diverges with no crash; S5/b vs. the foreign `Aevust/rincoin` fork shows the header/full-block asymmetry the mechanism is designed to produce (best-known-header can advance, validated tip does not). Full detail: [S5B/functional-test-results.md](#).

6 Scope: What This Testing Does Not Cover

Same deliberate scoping as S1 — not repeated in full here since none of it is scenario-specific. See [S1's own "Scope" section](#) for the full reasoning: header-level branch isolation, a dedicated P2P branch-declaration message, staged release engineering, live mainnet telemetry, and pool/seeder integration rehearsals are all out of scope or outstanding for the same reasons there.

7 Provenance

Repository	<code>rincoin-community/rincoin-core</code>
Branch	<code>consensus/s5b-testing</code>
Branched from	<code>consensus/s1-testing @ a8617fc73</code> (post <code>branch_id</code> migration)
Commit tested	<code>08faf8844</code>
CI run	github.com/rincoin-community/rincoin-core/actions/runs/33436296857 — in progress at time of writing
<code>branch_id</code>	<code>00112233445566778899aabbccddeeff</code> — the specification's own canonical synthetic test value (<code>technology/consensus-transition.md §5</code>), inherited directly, no separate migration needed
<code>scenario_id</code>	2 — provisional, ad-hoc, pending official upstream assignment (S1=1, S5/b=2, S6/b=3 by convention)
Build	testing-mode only; refuses to start on mainnet unless <code>RINCOIN_TESTING_ALLOW_MAINNET=1</code> is set in the process environment

8 Where to Go Next

- Full evidence, raw results, exact reproduction commands: [S5B/](#)
- The mechanism being tested: [technology/consensus-transition.md](#),
[technology/replay-protection-plan.md](#)
- The normative rule and its own vector set: [analysis/Rincoin_840k_S5B_Consensus_Change_Specif](#)
- Outstanding work tracked for maintainers: [S5B/open-items.md](#)

DRAFT